

Well-founded Semantics for Parametric Deductive Databases

Yann Loyer and Umberto Straccia

Via G. Moruzzi, 1 I-56124 Pisa (PI) ITALY
Number: 2002-TR-01¹

Abstract

Many frameworks of logic programming have been proposed to manage uncertain information in deductive databases and expert systems. Roughly, on the basis of how uncertainty is associated to facts and the rules in a program, they can be classified into *implication-based* (IB) and *annotation-based* (AB).

One fundamental issue that remains unaddressed in the IB approach is the representation and the manipulation of the non-monotonic mode of negation, an important feature for real applications.

Our focus in this paper is to introduce non-monotonic negation in the *parametric* IB framework, an unifying umbrella for IB frameworks. The semantical approach that we will adopt is based on the well-founded semantics, one of the most widely studied and used semantics of (classical) logic programs with negation.

ACM Categories and Subject Descriptors: F.4.1 [Mathematical Logic and Formal Languages]: Mathematical Logic - *Model theory*; I.2.3 [Artificial Intelligence]: Deduction and Theorem Proving - *Deduction*; I.2.4 [Artificial Intelligence]: Knowledge Representation Formalisms and Methods - *Representations*

1 Introduction

The management of uncertainty within deduction systems is an important issue in all those AI application domains in which the real world information to be represented is of imperfect nature (which is likely the rule rather than an exception). An impressive work has been carried out in the last decades, resulting in a number of concepts being investigated, a number of problems being identified and a number of solutions being developed (see, *e.g.* [1,6,17,29]).

¹ E-mail: straccia@iei.pi.cnr.it

First-Order Logic (FOL) has been the basis for most knowledge representation formalisms. Its basic units – individuals, their properties, and the relationship between them – naturally capture the way in which people encode their knowledge. Unfortunately, it is severely limited in its ability to represent our uncertainty about the world: a fact can only be known to be true, known to be false or neither. By contrast, most of our knowledge about the real world is not absolutely true. Additionally, practical considerations dictate that the framework used for knowledge representation with uncertainty admit efficient implementation and efficient computations. Logic database programming, with its advantage of modularity and its powerful top-down and bottom-up query processing techniques, has attracted the attention of researchers and numerous frameworks for deductive databases with uncertainty have been proposed [2,4,5,9,10,14–16,18–22,25–27,35–39].

The underlying uncertainty formalism in the proposed frameworks include probability theory [10,19,22,25–27,39], fuzzy set theory [2,35,37,38], multi-valued logic [9,15,16,20,21] and possibilistic logic [5]. These frameworks differ in (i) their underlying notion of uncertainty; (ii) the way in which uncertainties are manipulated; and (iii) the way in which uncertainty is associated with the facts and rules of a program. On the basis of (iii), these frameworks can be classified into *annotation based* (AB) and *implication based* (IB).

In the AB approach, a rule is of the form

$$A : f(\beta_1, \dots, \beta_n) \leftarrow B_1 : \beta_1, \dots, B_n : \beta_n \quad (1)$$

which asserts “the certainty of atom A is at least (or is in) $f(\beta_1, \dots, \beta_n)$, whenever the certainty of atom B_i is at least (or is in) β_i , $1 \leq i \leq n$ ”. Here f is an n -ary computable function and β_i is either a constant or a variable ranging over an appropriate certainty domain. Examples of AB frameworks include [15,16,25–27,36].

In the IB approach, a rule is of the form

$$A \stackrel{\alpha}{\leftarrow} B_1, \dots, B_n \quad (2)$$

which says that the certainty associated with the implication $B_1 \wedge \dots \wedge B_n \rightarrow A$ is α . Computationally, given an assignment v of certainties to the B_i s, the certainty of A is computed by taking the “conjunction” of the certainties $v(B_i)$ and then somehow “propagating” it to the rule head. Examples of the IB frameworks include [7,9,18–20,37]. It is not our aim to compare the two approaches in this paper. Refer to [20] for an exhaustive comparison. We limit our contribution in this sense to recall the following facts [20]: (i) while the way implication is treated in the AB approach is closer to classical logic, the

way rules are fired in the IB approach has a definite intuitive appeal and (ii) the AB approach is strictly more expressive than the IB. The down side is that query processing in the AB approach is more complicated, *e.g.* the fixpoint operator is not continuous in general, while it is in the IB approaches. From the above points, it is believed that the IB approach is easier to use and is more amenable for efficient implementation.

However, one fundamental issue that remains still unaddressed in the IB approach is the representation and the manipulation of the non-monotonic mode of *default negation*, *not A*, an without doubts important feature to be used in real applications. The major distinctive feature is that *not A* is assumed in the absence of sufficient evidence to the contrary. The meaning of “sufficient evidence” depends on the specific semantics used. Due to its importance, the problem of negation in logic programs has attracted many researchers and a broad variety of semantical approaches have been invented. For example, in Reiter’s *Closed World Assumption* [34], *not A* is assumed for atomic *A* if *A* is not provable, or, equivalently, if there is a minimal model in which *A* is false. On the other hand, in Minker’s *Generalised Closed World Assumption* [13,24], or in McCarthy’s *Circumscription* [23], *not A* is assumed only if *A* is false in *all* minimal models. In Clark’s *Predicate Completion* semantics for logic programs [3] this form of negation is called *negation-as-failure* because *not A* is derived whenever attempts to prove *A* finitely fail. The more recent semantics proposed for logic programs and deductive databases, such as the *stable semantics* [12], *well-founded semantics* [28], *partial stable* or *stationary semantics* [33] and *static semantics* [30] propose even more sophisticated meanings for default negation.

Contributions: We will extend the *parametric* IB framework [20], an unifying umbrella for IB frameworks, with default negation. The semantical approach that we will adopt is based on the well-founded semantics, one of the most widely studied and used semantics of (classical) logic programs with negation. From a semantics point of view, we will combine an alternating fix-point semantics similar to [11] with the fixpoint characterisation given in [20]. We will show that in case of positive programs, the *parametric* IB framework is obtained, while restricting logic programs to be Datalog programs, the classical well-founded semantics is obtained, *i.e.* our extension is a conservative extension.

The use of default negation has already been considered in some deductive databases with uncertainty frameworks. For instance, in [25], a stable semantics has been considered within the AB approach, but limited to the case where the underlying uncertainty formalism is probability theory. The stable semantics has been considered also in [38], where a semi-possibilistic logic has been proposed. In it, a particular negation operator has been introduced and

a fixed min/max-evaluation of conjunction and disjunction is adopted. To the best of our knowledge, there is no work dealing with default negation within the parametric IB approach.

In the following section, we will present the technical part of our contribution.

2 The well-founded semantics in parametric IB frameworks

2.1 Preliminaries

We recall the syntactical aspects of the parametric IB framework presented in [20] and extend it to negation.

Let L be an arbitrary first order language that contains infinitely many variable symbols, finitely many constants, and predicate symbols, but no function symbols. While L does not contain function symbols, it contains symbols for families of *propagation* (F_p), *conjunction* (F_c) and *disjunction* functions (F_d), called *combination functions*.

Let $\langle T, \preceq, \otimes, \oplus \rangle$ be a certainty lattice (a complete lattice) and $B(T)$ the set of finite multisets over T . With $\perp$ and $\top$ we denote the least and greatest element in T , respectively. A *propagation function* is a mapping from $T \times T$ to T and a *conjunction* or *disjunction* function is a mapping from $B(T)$ to T . Each kind of function must verify some of the following properties²:

- (1) monotonicity w.r.t. (with respect to) each one of its arguments;
- (2) continuity w.r.t. each one of its arguments;
- (3) bounded-above: $f(\alpha_1, \alpha_2) \preceq \alpha_i$, for $i = 1, 2, \forall \alpha_1, \alpha_2 \in T$;
- (4) bounded-below: $f(\alpha_1, \alpha_2) \succeq \alpha_i$, for $i = 1, 2, \forall \alpha_1, \alpha_2 \in T$;
- (5) commutativity: $f(\alpha_1, \alpha_2) = f(\alpha_2, \alpha_1), \forall \alpha_1, \alpha_2 \in T$;
- (6) associativity: $f(\alpha_1, f(\alpha_2, \alpha_3)) = f(f(\alpha_1, \alpha_2), \alpha_3), \forall \alpha_1, \alpha_2, \alpha_3 \in T$;
- (7) $f(\{\alpha\}) = \alpha, \forall \alpha \in T$;
- (8) $f(\emptyset) = \perp$;
- (9) $f(\emptyset) = \top$;
- (10) $f(\alpha, \top) = \alpha, \forall \alpha \in T$;

As postulated in [20]:

- (1) a conjunction function in F_c should satisfy properties 1, 2, 3, 5, 6, 7, 9 and 10;

² For simplicity, we formulate the properties treating any function as a binary function on T .

- (2) a propagation function in F_p should satisfy properties 1, 2, 3 and 10;
- (3) a disjunction function in F_d should satisfy properties 1, 2, 4, 5, 6, 7 and 8.

We also assume that there is a function from T to T , called *negation function* and denoted $\neg$, that is anti-monotone w.r.t. $\preceq$ and satisfies $\neg\neg\alpha = \alpha, \forall \alpha \in T$ and $\neg\perp = \top$.

Definition 1 (Normal parametric program) A normal parametric program P (*np-program*) is a 5-tuple $\langle T, R, C, P, D \rangle$, whose components are defined as follows:

- (1) T is a set of truth values partially ordered by $\preceq$. We assume that $\langle T, \preceq, \otimes, \oplus \rangle$ is a complete lattice, where $\otimes$ is the meet operator and $\oplus$ the join operator. We denote the least element of the lattice by $\perp$ and the greatest element by $\top$;
- (2) R is a finite set of normal parametric rules (*np-rules*), each of which is a statement of the form:

$$r : A \stackrel{\alpha_r}{\leftarrow} B_1, \dots, B_n, \neg C_1, \dots, \neg C_m$$

where A is an atomic formula and $B_1, \dots, B_n, C_1, \dots, C_m$ are atomic formulas or values in T and $\alpha_r \in T \setminus \{\perp\}$ is the certainty of the rule;

- (3) C is a mapping which associates with each np-rule a conjunction function in F_c ;
- (4) P is a mapping which associates with each np-rule a propagation function in F_p ;
- (5) D is a mapping which associates with each predicate symbol in P a disjunction function in F_d . ■

For ease of presentation, we write

$$r : A \stackrel{\alpha_r}{\leftarrow} B_1, \dots, B_n, \neg C_1, \dots, \neg C_m; \langle f_d, f_p, f_c \rangle$$

to represent a np-rule in which $f_d \in F_d$ is the disjunction function associated with the predicate symbol A and, $f_c \in F_c$ and $f_p \in F_p$ are respectively the conjunction and propagation functions associated with r . The intention is that the conjunction function (*e.g.* $\otimes$) determines the truth value of the conjunction of $B_1, \dots, B_n, \neg C_1, \dots, \neg C_m$, the propagation function (*e.g.* $\otimes$) determines how to “propagate” the truth value resulting from the evaluation of the body to the head, by taking into account the certainty α_r associated to the rule r , while the disjunction function (*e.g.* $\oplus$) dictates how to combine the certainties in case an atom is head of several rules. Note that in case negation is missing, np-programs are parametric programs (*p-programs*) as defined in [20].

We further define the *Herbrand base* HB_P of a np-program P as the set of all instantiated atoms corresponding to atoms appearing in P and define P^* to

be the *Herbrand instantiation* of P , i.e. the set of all ground instantiations of the rules in P .

Example 1 Consider the complete lattice $\langle T, \preceq, \otimes, \oplus \rangle$, where T is $[0, 1]$, $\forall, a, b \in [0, 1], a \preceq b$ iff $a \leq b$, $a \otimes b = \min(a, b)$, and $a \oplus b = \max(a, b)$. Consider, the two disjunction functions $f_d^1(\alpha, \beta) = \max(\alpha, \beta)$ and $f_d^2(\alpha, \beta) = \alpha + \beta - \alpha \cdot \beta$. $f_p(\alpha, \beta) = \alpha \cdot \beta$. The negation function is the usual function $\neg(\cdot) = 1 - (\cdot)$. Then the following are two programs³ $P_i, i = 1, 2$:

$$\begin{aligned} A &\stackrel{1}{\leftarrow} A; \langle f_d^i, f_p, A \cdot A \rangle \\ A &\stackrel{1}{\leftarrow} B; \langle f_d^i, f_p, B \rangle \\ B &\stackrel{1}{\leftarrow} A, \neg C; \langle f_d^i, f_p, A \cdot \neg C \rangle \\ C &\stackrel{1}{\leftarrow} \neg B; \langle f_d^i, f_p, \neg B \rangle \\ C &\stackrel{1}{\leftarrow} 0.2; \langle f_d^i, f_p, 0.2 \rangle \\ B &\stackrel{1}{\leftarrow} 0.3; \langle f_d^i, f_p, 0.3 \rangle \end{aligned}$$

□

2.2 Interpretations of programs

An interpretation of a np-program P is a function that assigns to all atoms of the Herbrand base of P a value in T . We denote $V_P(T)$ the set of all interpretations of P .

An important issue is to determine which is the intended meaning or semantics of a np-program. Following the usual approach, the semantics of a program P is determined by selecting a particular interpretation of P in the set of models of P . In logic programs without negation, as well as in the parametric IB framework, that chosen model is usually the least model of P w.r.t. $\preceq$.

Introducing negation in classical logic programs, and in particular in our parametric IB framework, some np-programs do not have an unique minimal model, as shown in the following examples.

Example 2 Assume that our lattice is the classical boolean logics and let P

³ We have made explicit the conjunction functions. For instance, $A \cdot A$ means that we take as conjunction the square of the evaluation of A .

be the program defined by the two rules

$$\begin{aligned} A &\stackrel{t}{\leftarrow} \neg B \\ B &\stackrel{t}{\leftarrow} \neg A \end{aligned}$$

The program P has two minimal models: I_1 that assigns to A the value false and to B the value true, and I_2 that assigns to A the value true and to B the value false. I_1 and I_2 are not comparable w.r.t. the truth ordering. $\square$

Example 3 Let T be $[0, 1]$. Consider, as usual, $f_c(\alpha, \beta) = \min(\alpha, \beta)$, $f_d(\alpha, \beta) = \max(\alpha, \beta)$, $f_p(\alpha, \beta) = \alpha \cdot \beta$ and the usual negation function. Consider the program P :

$$\begin{aligned} A &\stackrel{1}{\leftarrow} \neg B; \langle f_d^i, f_p, \neg B \rangle \\ B &\stackrel{1}{\leftarrow} \neg A; \langle f_d^i, f_p, \neg A \rangle \\ A &\stackrel{1}{\leftarrow} 0.2; \langle f_d^i, f_p, 0.2 \rangle \\ B &\stackrel{1}{\leftarrow} 0.3; \langle f_d^i, f_p, 0.3 \rangle \end{aligned}$$

This program will have an infinite number of models (those in the grey zone in Figure 1). There are also an infinite number of minimal models (those on the bold line) according to the order $(a, b) \preceq (c, d)$ iff $a \preceq c$ and $b \preceq d$. $\square$

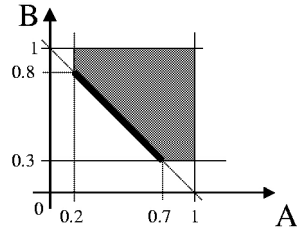


Fig. 1. Infinite minimal models.

An usual way to deal with such situations consists in introducing a third truth value for atoms, *undefined* or *unknown*, and by considering as (well-founded)

semantics of P the interpretation that assigns that value undefined to both A and to B .

Rather introducing a new truth value, we will go around that by dealing with *partial interpretations*: partial interpretations correspond to interpretations that assign values only to some atoms of HB_P and are not defined for the other atoms. An atom that is not defined in a partial interpretation can be seen as an underdefined atom: that atom may have a value but whose value is currently unknown.

Definition 2 (Partial interpretation) *Let P be a np-program. A partial interpretation I of P is a set $\{A : \mu \mid A \in HB_P \text{ and } \mu \in T\}$.* ■

A partial interpretation I can be seen as a function defined by: for all ground atoms A , if $A : \mu \in I$ then $I(A) = \mu$ else $I(A)$ is not defined. Of course, an interpretation can be seen as a partial interpretation. Interpretations and partial interpretations will be used as functions or as sets following the context.

In the following, given a np-program P , (i) we denote with r_A a rule $(r : A \stackrel{\alpha_r}{\leftarrow} B_1, \dots, B_n, \neg C_1, \dots, \neg C_m; \langle f_d, f_p, f_d \rangle) \in P^*$, whose head is A ; and (ii) given an interpretation I such that each premise in the body of r_A is defined under I , with $I(r_A)$ we denote the evaluation of the body of r_A w.r.t. I , i.e.

$$I(r_A) = f_p(\alpha_r, f_c(\{I(B_1), \dots, I(B_n), \neg I(C_1), \dots, \neg I(C_m)\}))$$

(iii) $I(r_A)$ is undefined in case some premise in the body is undefined in I , except for the case where there is an i such that $I(B_i) = \perp$ or $I(C_i) = \top$. In that case, we define $I(r_A) = \perp$.

Definition 3 (Satisfaction of a np-program) *Let P be a np-program and let I be a partial interpretation of P . Then we say that I satisfies (is a model of) P , denoted $\models_I P$, iff $\forall A \in HB_P$:*

- (1) *if there is a rule $r_A \in P^*$ such that $I(r_A) = \top$, then $I(A) = \top$;*
- (2) *if for some rule $r_A \in P^*$, $I(r_A)$ is undefined and case 1. does not hold, then $I(A)$ is undefined or $I(A) = \top$;*
- (3) *if for all rules $r_A \in P^*$, $I(r_A)$ is defined, then $I(A) \succeq f_d(X)$, where $X = \{I(r_A) : r_A \in P^*\}$. f_d is the disjunction function associated with $\pi(A)$, the predicate symbol of A .* ■

Example 4 *Consider program P_1 in Example 1. It can be verified that the interpretation I_1 such that $I_1(A) = 1$ and both $I_1(B)$ and $I_1(C)$ undefined is a model of P_1 . Similarly, the interpretation I_2 ($I_2(A) = I_2(B) = 0.3$ and $I_2(C) = 0.7$) is a model of P_1 . More involved is to show that for program P_2 in Example 1, the interpretation I_3 ($I_3(A) = 1, I_3(B) = 0.68\overline{18}$ and $I_3(C) = 0.\overline{45}$) is a model of P_2 .*

For the program P in Example 2, it is easily verified that the interpretations I_4 ($I_4(A)$ and $I_4(B)$ undefined), I_5 ($I_5(A) = \text{true}$ and $I_5(B) = \text{false}$) and I_6 ($I_6(A) = \text{false}$, and $I_6(B) = \text{true}$) are models of P .

Finally, for the program P in Example 3, the interpretations I_x^y such that $0.2 \leq x \leq 1$, $0.3 \leq y \leq 1$, $y \geq 1 - x$, $I_x^y(A) = x$ and $I_x^y(B) = y$ are all models of P . Note that the interpretation I_7 undefined on A and B is a model of P as well. $\square$

It is worth noting that if we restrict our attention to positive programs only, then the definition of satisfiability of np-programs reduces to that of satisfiability of p-programs defined in [20], where the interpretation I is not partial but defined for all atoms in $HB(P)$.

2.3 Alternating fixpoint and compromise semantics

In this section we will define our well-founded semantics for np-programs by determining which, among all the models, is the intended model of a np-program.

First, we extend the ordering on T to the space of interpretations $V_P(T)$. Let I_1 and I_2 be in $V_P(T)$, then $I_1 \preceq I_2$ if and only if $I_1(A) \preceq I_2(A)$ for all ground atoms A . Under this ordering $V_P(T)$ becomes a complete lattice, and we have $(I_1 \otimes I_2)(A) = I_1(A) \otimes I_2(A)$, and similarly for the other operators. The actions of functions can be extended from atoms to formulas as follows: $I(f_c(X, Y)) = f_c(I(X), I(Y))$, and similarly for the other functions. Finally, for all α in T and for all I in $V_P(T)$, $I(\alpha) = \alpha$.

We now define a new operator T_P inspired by [8,30–32]. It infers new information from two interpretations: the first one is used to evaluate the positive literals, while the second one is used to evaluate the negative literals of the bodies of rules in P .

Definition 4 (Immediate consequence operator) *Let P be any np-program. The immediate consequence operator T_P is a mapping from $V_P(T) \times V_P(T)$ to $V_P(T)$, defined as follows: for every pair (I, J) of interpretations in $V_P(T)$, for every atom A , $T_P(I, J)(A) = f_d(X)$, where f_d is the disjunction function associated with $\pi(A)$, the predicate symbol of A , and*

$$X = \{f_p(\alpha_r, f_c(\{I(B_1), \dots, I(B_n), \neg J(C_1), \dots, \neg J(C_m)\})) : \\ (r : A \stackrel{\alpha_r}{\leftarrow} B_1, \dots, B_n, \neg C_1, \dots, \neg C_m; \langle f_d, f_p, f_d \rangle) \in P^*\}$$

■

Note that the T_P operator applies to interpretations only and not to partial interpretations. Additionally, the interpretation $T_P(I, J)$ is in $V_P(T)$. Furthermore, in case negation is absent, T_P reduces to the immediate consequence operator defined in [20].

It is easy to prove that

Proposition 1 *Let P be any np-program. T_P is monotonic in its first argument, and anti-monotone in its second argument w.r.t. $\preceq$. ■*

Using Proposition 1 and the Knaster-Tarsky theorem, we can define an operator S_P , inspired from [11] and derived from T_P , that takes an interpretation J as input, first evaluates the negative literals of the program w.r.t. J , and then returns the minimal model of the resulting “positive” np-program w.r.t. $\preceq$.

Let $I_\perp$ be the interpretation that assigns the value $\perp$ to all atoms of $HB(P)$, i.e. the minimal element of $V_P(T)$ w.r.t. $\preceq$, while let $I_\top$ be the interpretation that assigns the value $\top$ to all atoms of $HB(P)$, i.e. the maximal element of $V_P(T)$ w.r.t. $\preceq$.

Definition 5 (Alternate consequence operator) *Let P be any np-program. We define $S_P(J) = T_P^\infty(I_\perp, J)$, i.e. the least fixpoint of T_P w.r.t. $\preceq$ for a given interpretation J . ■*

Intuitively, $S_P(J)$ is the interpretation that assigns to negative literals the evaluation of them in the fixed interpretation J and then applies the usual T_P operator. Note that the closure ordinal is at most ω .

From Proposition 1, we easily derive the following property of S_P .

Proposition 2 *Let P be any np-program, then S_P is anti-monotone w.r.t. $\preceq$ and, thus $S_P \circ S_P$ is monotone. ■*

There is a well-know property, which derives from the Knaster-Tarski theorem and deals with anti-monotone functions on complete lattices:

Proposition 3 ([40]) *Suppose that a function f is anti-monotone on a complete lattice T . Then there are unique two elements μ and ν of T , called extreme oscillation points of f , such that the following hold:*

- μ and ν are the least and greatest fixpoint of $f \circ f$ (i.e. of f composed with f);
- f oscillates between μ and ν in the sense that $f(\mu) = \nu$ and $f(\nu) = \mu$;
- if x and y are also elements of T between which f oscillates then x and y lie between μ and ν . ■

Under the ordering $\preceq$, S_P is anti-monotone and $V_P(T)$ is a complete lattice, so S_P has two extreme oscillation points under this ordering.

Proposition 4 *Let P be any np-program, S_P has two extreme oscillation points under $\preceq$, $S_P^\perp = (S_P \circ S_P)^\infty(I_\perp)$ and $S_P^\top = (S_P \circ S_P)^\infty(I_\top)$, with $S_P^\perp \preceq S_P^\top$. ■*

As in Van Gelder's alternated fixpoint approach [11], $S_P^\perp$ and $S_P^\top$ are respectively an under-estimation and an over-estimation of P . As the meaning of P , we propose to consider the consensus or compromise between those two interpretations, *i.e.* to consider as defined only the atoms whose values coincide in both limit interpretations. We define the *compromise* between $S_P^\perp$ and $S_P^\top$ to be the intersection between $S_P^\perp$ and $S_P^\top$. It can be shown that

Proposition 5 *Let P be any np-program, then $\models_{S_P^\perp \cap S_P^\top} P$. ■*

Therefore, the interesting property of the compromise semantics is that it defines a model and this model will be the intended meaning or semantics of np-programs.

Definition 6 (Compromise semantics) *Let P be any np-program, the compromise semantics of P , $CS(P)$, is defined by $CS(P) = S_P^\perp \cap S_P^\top$. ■*

Example 5 *Consider program P_1 and P_2 of Example 1. It can be verified that $CS(P_1) = I_2$, while $CS(P_2) = I_3$.*

For the program P in Example 2, $CS(P) = I_4$, while for the program P in Example 3, $CS(P) = I_7$. □

Example 6 *Consider the following np-program P on $T = [0, 1]$, with the usual disjunction function $f_d(\alpha, \beta) = \max(\alpha, \beta)$, the conjunction function $f_c(\alpha, \beta) = \min(\alpha, \beta)$ and the propagation function $f_p = f_c$. Negation is as usual. All rules have the same functions associated and the same weight 1, so*

we omit all of them for readability.

$$\begin{aligned}
A &\leftarrow B, \neg C; \\
B &\leftarrow E; \\
D &\leftarrow C; \\
F &\leftarrow \neg D, E; \\
G &\leftarrow A, C; \\
H &\leftarrow \neg G; \\
L &\leftarrow \neg E; \\
M &\leftarrow \neg L; \\
A &\leftarrow \neg C, H; \\
B &\leftarrow \neg D, E; \\
E &\leftarrow 0.2;
\end{aligned}$$

It can be shown that $CS(P) = I$, where I is defined as follows:

	A	B	C	D	E	F	G	H	L	M
I	1	0.2	0	0	0.2	0.2	0	1	0.8	0.2

□

We conclude this section by showing that our extension is a conservative one.

Our semantics extends the Lakshmanan and Shiri's semantics of parametric programs presented in [20] to normal parametric programs. This is due to the fact that the machinery developed in order to deal with negation has no effect on positive programs.

Proposition 6 *If P is a np-program without negation then the compromise semantics $CS(P)$ of P coincides with the Lakshmanan and Shiri's semantics of P .* ■

Finally, we compare our semantics with the well-founded semantics of classical logic programs defined in [28]. It can be verified that

Proposition 7 *Let P be a Datalog program with negation. The compromise semantics $CS(P)$ of P coincides with the well-founded semantics of P .* ■

Example 7 *Consider the np-program P as in Example 6 except that the rule*

$E \leftarrow 0.2$ is replaced with $E \leftarrow 1$. Then the interpretation $CS(P) = I$, where I is defined as

	A	B	C	D	E	F	G	H	L	M
I	1	1	0	0	1	1	0	1	0	1

corresponds exactly to the well-founded semantics of the program, which is $I = \{A, B, \neg C, \neg D, E, F, \neg G, H, \neg L, M\}$. $\square$

3 Conclusions

We have extended the parametric IB approach [20], a general framework for the representation and the manipulation of uncertainty in logic databases and expert systems, with default negation. The semantical approach that we adopted for default negation in logic programs is based on the well-founded semantics, a widely studied and used semantics for classical logic programs with negation. Technically, we integrated the fixpoint semantics of the parametric IB approach with the alternating fixpoint techniques proposed in [11]. We have also shown that our extension is a conservative extension.

References

- [1] Fahiem Bacchus. *Representing and Reasoning with Probabilistic Knowledge*. The MIT Press, 1990.
- [2] True H. Cao. Annotated fuzzy logic programs. *Fuzzy Sets and Systems*, 113(2):277–298, 2000.
- [3] K.L. Clark. On closed world data bases. In Hervé Gallaire and Jack Minker, editors, *Logic and data bases*, pages 293–322. Plenum Press, New York, NY, 1978.
- [4] Alex Dekhtyar and V.S. Subrahmanian. Hybrid probabilistic programs. In *Proc. of the 13th Int. Conf. on Logic Programming (ICLP-97)*, Leuven, Belgium, 1997. The MIT Press.
- [5] Didier Dubois, Jérôme Lang, and Henri Prade. Towards possibilistic logic programming. In *Proc. of the 8th Int. Conf. on Logic Programming (ICLP-91)*, pages 581–595. The MIT Press, 1991.
- [6] Didier Dubois and Henri Prade. Approximate and commonsense reasoning: From theory to practice. In Zbigniew W. Ras and Michalewicz Maciek, editors,

Proc. of the 9th Int. Sym. on Methodologies for Intelligent Systems (ISMIS-96), number 1079 in Lecture Notes in Artificial Intelligence, pages 19–33. Springer-Verlag, 1996.

- [7] Gonzalo Escalada-Imaz and Felip Manyà. Efficient interpretation of propositional multiple-valued logic programs. In *Proc. of the 5th Int. Conf. on Information Processing and Managment of Uncertainty in Knowledge-Based Systems, (IPMU-94)*, number 945 in Lecture Notes in Computer Science, pages 428–439. Springer-Verlag, 1994.
- [8] M. C. Fitting. The family of stable models. *Journal of Logic Programming*, 17:197–225, 1993.
- [9] Melvin Fitting. Bilattices and the semantics of logic programming. *Journal of Logic Programming*, 11:91–116, 1991.
- [10] Norbert Fuhr. Probabilistic datalog: Implementing logical information retrieval for advanced applications. *Journal of the American Society for Information Science*, 51(2):95–110, 2000.
- [11] Allen Van Gelder. The alternating fixpoint of logic programs with negation. In *Proc. of the 8th ACM SIGACT SIGMOD Sym. on Principles of Database Systems (PODS-89)*, pages 1–10, 1989.
- [12] M. Gelfond and V. Lifschitz. Logic programs with classical negation. In David H. D. Warren and Peter Szeredi, editors, *Proceedings of the Seventh International Conference on Logic Programming*, pages 579–597, Jerusalem, 1990. The MIT Press.
- [13] Michael Gelfond, Halina Przymusinska, and Teodor C. Przymusinski. On the relationship between circumscription and negation as failure. *Artificial Intelligence*, 38:75–94, 1989.
- [14] Mitsuru Ishizuka and Naoki Kanai. Prolog-ELF: incorporating fuzzy logic. In *Proc. of the 9th Int. Joint Conf. on Artificial Intelligence (IJCAI-85)*, pages 701–703, Los Angeles, CA, 1985.
- [15] M. Kifer and Ai Li. On the semantics of rule-based expert systems with uncertainty. In *Proc. of the Int. Conf. on Database Theory (ICDT-88)*, number 326 in Lecture Notes in Computer Science, pages 102–117. Springer-Verlag, 1988.
- [16] Michael Kifer and V.S. Subrahmanian. Theory of generalized annotated logic programming and its applications. *Journal of Logic Programming*, 12:335–367, 1992.
- [17] R. Kruse, E. Schwecke, and J. Heinsohn. *Uncertainty and Vagueness in Knowledge Based Systems*. Springer-Verlag, Berlin, Germany, 1991.
- [18] Laks Lakshmanan. An epistemic foundation for logic programming with uncertainty. In *Foundations of Software Technology and Theoretical Computer Science*, number 880 in Lecture Notes in Computer Science, pages 89–100. Springer-Verlag, 1994.

- [19] Laks V.S. Lakshmanan and Nematollaah Shiri. Probabilistic deductive databases. In *Int'l Logic Programming Symposium*, pages 254–268, 1994.
- [20] Laks V.S. Lakshmanan and Nematollaah Shiri. A parametric approach to deductive databases with uncertainty. *IEEE Transactions on Knowledge and Data Engineering*, 13(4):554–570, 2001.
- [21] James J. Lu. Logic programming with signs and annotations. *Journal of Logic and Computation*, 6(6):755–778, 1996.
- [22] Thomas Lukasiewicz. Probabilistic logic programming. In *Proc. of the 13th European Conf. on Artificial Intelligence (ECAI-98)*, pages 388–392, Brighton (England), August 1998.
- [23] John McCarthy. Circumscription - a form of nonmonotonic reasoning. *Artificial Intelligence*, 13:27–39, 1980.
- [24] Jack Minker. On indefinite data bases and the closed world assumption. In Springer-Verlag, editor, *Proc. of the 6th Conf. on Automated Deduction (CADE-82)*, number 138 in Lecture Notes in Computer Science, 1982.
- [25] Raymond Ng and V.S. Subrahmanian. Stable model semantics for probabilistic deductive databases. In Zbigniew W. Ras and Maria Zemenkova, editors, *Proc. of the 6th Int. Sym. on Methodologies for Intelligent Systems (ISMIS-91)*, number 542 in Lecture Notes in Artificial Intelligence, pages 163–171. Springer-Verlag, 1991.
- [26] Raymond Ng and V.S. Subrahmanian. Probabilistic logic programming. *Information and Computation*, 101(2):150–201, 1993.
- [27] Raymond Ng and V.S. Subrahmanian. A semantical framework for supporting subjective and conditional probabilities in deductive databases. *Journal of Automated Reasoning*, 10(3):191–235, 1993.
- [28] Allen nva Gelder, Kenneth A. Ross, and John S. Schlimpf. The well-founded semantics for general logic programs. *Journal of the ACM*, 38(3):620–650, January 1991.
- [29] J. Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, Los Altos, 1988.
- [30] T. Przymusiński. Static semantics for normal and disjunctive logic programs. *Annals of Mathematics and Artificial Intelligence*, 14:323–357, 1995.
- [31] T. C. Przymusiński. Extended stable semantics for normal and disjunctive programs. In D. H. D. Warren and P. Szeredi, editors, *Proceedings of the Seventh International Conference on Logic Programming*, pages 459–477. MIT Press, 1990.
- [32] T. C. Przymusiński. Stationary semantics for disjunctive logic programs and deductive databases. In S. Debray and H. Hermenegildo, editors, *Logic Programming, Proceedings of the 1990 North American Conference*, pages 40–59. MIT Press, 1990.

- [33] Teodor C. Przymusiński. The well-founded semantics coincides with the three-valued stable semantics. *Fundamenta Informaticae*, 13(4):445–463, 1990.
- [34] Raymond Reiter. On closed world data bases. In Hervé Gallaire and Jack Minker, editors, *Logic and data bases*, pages 55–76. Plenum Press, New York, NY, 1978.
- [35] Ehud Y. Shapiro. Logic programs with uncertainties: A tool for implementing rule-based systems. In *Proc. of the 8th Int. Joint Conf. on Artificial Intelligence (IJCAI-83)*, pages 529–532, 1983.
- [36] V.S. Subramanian. On the semantics of quantitative logic programs. In *Proc. 4th IEEE Symp. on Logic Programming*, pages 173–182. Computer Society Press, 1987.
- [37] M.H. van Emden. Quantitative deduction and its fixpoint theory. *Journal of Philosophical Logic*, (1):37–53, 1986.
- [38] Gerd Wagner. Negation in fuzzy and possibilistic logic programs. In T. Martin and F. Arcelli, editors, *Logic programming and Soft Computing*, pages –. Research Studies Press, 1998.
- [39] Beat Wütrich. Probabilistic knowledge bases. *IEEE Transactions on Knowledge and Data Engineering*, 7(5):691–698, 1995.
- [40] S. Yablo. Truth and reflection. *Journal of Philosophical Logic*, 14:297–349, 1985.